Capítulo 3. Estrategia y resultados de pruebas

Redacción	Edgar Alexis Sánchez Huerta (Encargado de Pruebas)
Revisión cruzada	Cristian Ibrahim Salomon Marquez (Desarrollador CRUD) — pendiente de firma
Fecha de las ejecuciones	5 de noviembre de 2026

El presente capítulo describe cómo se comprobó que el sistema de Rutas del Bajío cumple las reglas de negocio documentadas en los capítulos anteriores. Se presenta el entorno de pruebas, la matriz unificada de criterios con su trazabilidad hacia las pruebas automatizadas, los resultados de las ejecuciones reales, la evaluación de la calidad de la suite mediante la técnica de mutación, los defectos encontrados y los límites de lo probado. Todas las cifras provienen de ejecuciones de `pytest` sobre el repositorio; las salidas completas se conservan en `docs/evidencias/` y sus capturas en el Anexo C.

3.1 Entorno de pruebas

Herramientas y configuración

Las pruebas se escribieron con `pytest` 9.1.1 sobre Python 3.14.6. La biblioteca se instaló en un entorno virtual del proyecto (`.venv/`), excluido del repositorio mediante `.gitignore`, de modo que la versión empleada no depende de la instalación global de cada equipo. El archivo `pytest.ini` fija tres decisiones: `testpaths = tests` (solo se recolectan pruebas de esa carpeta), `pythonpath = .` (el paquete `rutas_bajio` se importa sin instalarlo) y `addopts = -ra --strict-markers`. La opción `--strict-markers` convierte en error cualquier marcador no registrado; sin ella, un marcador mal escrito (por ejemplo `integracon`) dejaría pruebas fuera de una ejecución filtrada sin que nadie lo advirtiera.

El código del cuaderno `Desarrollo.ipynb` no puede importarse desde `pytest`, por lo que sus celdas de clases se copiaron sin cambios al paquete `rutas_bajio/` (`modelos.py` con los validadores y las entidades `Cliente`, `Ruta` y `Pedido`; `gestor.py` con `GestorRutasBajio`). La alternativa descartada fue probar el cuaderno con herramientas de ejecución de celdas, que vuelven las pruebas dependientes del orden de ejecución del cuaderno.

Estructura de la carpeta de pruebas

La Tabla 3.1 contrasta el árbol propuesto en el Acta del Encargado de Pruebas (véase Anexo B) con lo implementado. Se eligió un archivo por área de comportamiento y no un archivo por clase, porque el sistema tiene un solo gestor que concentra la mayor parte de la lógica y un archivo por clase habría producido un archivo de pruebas desproporcionado.

Tabla 3.1: Árbol de pruebas planeado frente a lo implementado.

Archivo	Contenido	Casos	Diferencia con lo planeado
`conftest.py`	Fixtures compartidas	—	Sin cambios
`test_entorno.py`	Verificación de que las fixtures arrancan	1	No estaba en el acta; se agregó como prueba de humo
`test_validadores.py`	C-03, C-04, C-07	12	Sin cambios
`test_modelos.py`	C-01, C-02, C-05, C-06, C-08	14	Sin cambios
`test_csv.py`	C-09, C-10, C-12, C-13, C-14, C-19, C-33	8	Se agregó C-33 (descubierto por mutación)
`test_gestor_crud.py`	C-15, C-21, C-25, C-26, C-27, C-32, C-34	9	Se agregaron C-32 y C-34 (mutación)
`test_relaciones.py`	C-16, C-17, C-18, C-20	6	C-19 se movió a `test_csv.py` porque escribe un CSV
`test_estados.py`	C-24, C-28 a C-31	18	Absorbe los criterios de estados del Acta del Documentador
`test_integracion.py`	C-11, C-22, C-23	3	Sin cambios
`mutaciones.py` (raíz)	Script de la técnica de mutación	—	No estaba en el acta; ver apartado 3.4

Fuente: elaboración propia a partir del repositorio del equipo.

El Acta del Documentador proponía además cinco archivos (`test_transiciones.py`, `test_limpieza.py`, `test_lote.py`, `test_lote_archivo.py` y `test_persistencia.py`) y los marcadores `estados` y `lote`. Los criterios de transiciones se integraron en `test_estados.py`, porque prueban el mismo método (`cambiar_estado`) y separarlos habría duplicado la preparación de datos. Los archivos de lote no se crearon porque el componente que debían probar, `rutas_bajio/lotes.py`, no llegó a implementarse (véase el apartado 3.6); por la misma razón los marcadores `estados` y `lote` no se registraron en `pytest.ini`.

Fixtures compartidas y aislamiento de los datos

Las fixtures de `conftest.py` (Tabla 3.2) evitan repetir la preparación del gestor en cada prueba. La decisión más importante es que **ninguna prueba usa la carpeta real de datos**: `GestorRutasBajio` escribe por defecto en `datos_notebook/`, y si una prueba la usara podría borrar o alterar datos de trabajo, además de depender de lo que otra prueba dejó escrito. Por ello la fixture `directorio` construye la ruta a partir de `tmp_path`, una carpeta temporal que `pytest` crea vacía para cada prueba y elimina después. Cada prueba es así independiente del orden de ejecución y repetible en cualquier equipo. La alternativa descartada, versionar CSV de ejemplo en el repositorio, obliga a mantener archivos compartidos y oculta el dato que causa cada fallo; con `tmp_path`, los CSV defectuosos se escriben dentro de la propia prueba y quedan a la vista.

Tabla 3.2: Fixtures definidas en `tests/conftest.py`.

Fixture	Qué entrega	Usada por
`directorio`	`tmp_path / "datos"`, carpeta temporal para los CSV	Pruebas que recargan o escriben CSV
`gestor`	`GestorRutasBajio` vacío sobre `directorio`	CRUD, CSV, integración
`cliente`	`CLI-010`, Constructora Guanajuato S.A., Guanajuato Capital	CRUD, CSV, integración
`ruta`	`RUTA-03` León → Guanajuato Capital, Julián Pérez	`gestor_con_datos`
`gestor_con_datos`	2 clientes, 3 rutas (`RUTA-02` no disponible) y `PED-0001` pendiente con `RUTA-03`	Relaciones, estados, CRUD

Fuente: elaboración propia.

Las pruebas siguen la convención `test_<componente>_<escenario>` y llevan en el docstring el identificador del criterio que cubren (por ejemplo `"""C-15"""`), lo que permite rastrear cada prueba hasta la matriz. Los criterios con varios datos de entrada (C-01, C-02, C-03, C-04, C-07, C-24, C-29, entre otros) se implementaron con `@pytest.mark.parametrize` y `pytest.param(..., id=...)`, de modo que el reporte muestra cada caso por su nombre (`[telefono_16_digitos]`) en lugar de un índice. Se registraron dos marcadores: `csv`, para las pruebas que leen o escriben archivos, e `integracion`, para las que combinan varias operaciones y recargan el gestor.

3.2 Matriz unificada de criterios de prueba

Unificación de las dos actas

Durante el parcial se elaboraron dos matrices de criterios: la del Acta del Encargado de Pruebas (sesión N3, 27 criterios, C-01 a C-27) y la Tabla 3 del Acta del Documentador (19 criterios sobre

estados y carga por lote, también numerados C-01 a C-19). Ambas reutilizaban los mismos identificadores para criterios distintos, lo que habría vuelto ambigua cualquier referencia en el reporte. Se decidió conservar la numeración del Acta del Encargado de Pruebas, porque ya estaba escrita en los docstrings de las pruebas, y renumerar los criterios del Acta del Documentador a continuación. Los criterios que verificaban lo mismo se fusionaron en lugar de duplicarse. La Tabla 3.3 registra la correspondencia; en adelante, todo el documento usa solo los identificadores unificados.

Tabla 3.3: Correspondencia entre la Tabla 3 del Acta del Documentador (A2) y la matriz unificada.

Acta del Documentador	Matriz unificada	Tratamiento
A2 C-01	C-23 y C-28	Fusionado: las transiciones `pendiente → en_transito → entregado` ya estaban en C-23; `pendiente → cancelado` se separó como C-28
A2 C-02	C-24	Fusionado: se agregaron a C-24 los casos `en_transito → pendiente` y `pendiente → pendiente`
A2 C-03, C-04, C-05	C-29, C-30, C-31	Renumerados
A2 C-17	C-23	Fusionado: C-23 ya verifica que el estado persiste al recargar
A2 C-06 a C-16, C-18, C-19	C-35 a C-47	Renumerados; componente no implementado
— (técnica de mutación)	C-32, C-33, C-34	Criterios nuevos descubiertos en el apartado 3.4

Fuente: elaboración propia a partir de las actas originales (Anexo B).

Matriz

La Tabla 3.4 presenta la matriz unificada en el formato del Anexo A.4. La columna *Prueba* indica el archivo de `tests/` y el número de casos ejecutados; el nombre completo de cada función aparece en la Tabla 3.5. La prioridad (A alta, M media, B baja) se asignó según el daño al proceso de negocio: un dato perdido, una relación rota, un identificador duplicado o un estado ilegal son de prioridad alta.

Tabla 3.4: Matriz unificada de criterios de prueba con trazabilidad.

ID	Componente	Criterio	Tipo	Escenario y datos	Resultado esperado	Pr.	Prueba (casos)	Resultado
C-01	`clave` / `Cliente`	Rechaza claves mal formadas	Error	`CLI-1`, vacía, `PED-0001`, `cli`	`ValueError` en cada caso	M	modelos (4)	Pasa 4/4
C-02	`Cliente`	Teléfono de 10 a 15 dígitos	Límite	9, 10, 15 y 16 dígitos	9 y 16 rechazados; 10 y 15 aceptados	M	modelos (4)	Pasa 4/4
C-03	`validar_ciudad`	Normaliza y rechaza ciudades fuera del catálogo	Error	`león`, `CELAYA`, `Morelia`	`León`, `Celaya`; `Morelia` → `ValueError`	M	validadores (3)	Pasa 3/3
C-04	`entero_positivo`	Cantidad entera mayor que cero	Error	0, −5, `muchas`, `3.5`, `True`; `"40"`	`ValueError`; `"40"` → 40	A	validadores (6)	Pasa 6/6
C-05	`Ruta` / `Pedido`	Origen y destino distintos	Error	León → León	`ValueError` en ambas clases	M	modelos (2)	Pasa 2/2
C-06	`Pedido`	En tránsito exige ruta	Error	`en_transito`, sin ruta	`ValueError`	A	modelos (1)	Pasa 1/1
C-07	`fecha_iso`	Formato y calendario válidos	Error	`2026-02-30`, `15/09/2026`, vacía	`ValueError`	M	validadores (3)	Pasa 3/3
C-08	`to_dict` / `from_dict`	Ida y vuelta conserva valores	Persist.	Pedido sin y con ruta; ruta no disponible	Diccionarios iguales	A	modelos (3)	Pasa 3/3
C-09	`cargar`	Arranque sin archivos	Límite	Carpeta vacía	Listas vacías, `ultimo_folio == 0`	A	csv (1)	Pasa 1/1
C-10	`cargar`	CSV con solo encabezado	Límite	`clientes.csv` de una línea	`clientes == []`	M	csv (1)	Pasa 1/1
C-11	`crear_*` + `cargar`	Lo guardado vuelve igual y en orden	Persist.	Cliente, ruta y dos pedidos; segundo gestor	Mismos registros, mismo orden	A	integracion (1)	Pasa 1/1
C-12	`_leer`	Rechaza encabezado incorrecto y fila incompleta	Error	Encabezado `id,nombre`; fila corta	`ValueError` con mensaje	M	csv (2)	Pasa 2/2
C-13	`guardar` / `_leer`	Comas y acentos sobreviven	Persist.	`Aceros, Fundición del Bajío`	Mismo nombre al recargar	M	csv (1)	Pasa 1/1
C-14	`_escribir`	No deja archivos `.tmp`	Persist.	Crear un cliente	Sin `*.tmp`	B	csv (1)	Pasa 1/1
C-15	`crear_cliente`	Rechaza id duplicado sin tocar el archivo	Error	`CLI-010` dos veces	`ValueError`; CSV idéntico	A	gestor_crud (1)	Pasa 1/1
C-16	`crear_pedido`	Cliente inexistente	Relación	`CLI-404`	`ValueError`; sin pedidos	A	relaciones (1)	Pasa 1/1
C-17	`crear_pedido` / `asignar_ruta`	Ruta del mismo trayecto	Relación	León → Irapuato con `RUTA-03`	`ValueError`; no se asigna	A	relaciones (2)	Pasa 2/2
C-18	`crear_pedido`	Ruta no disponible	Relación	`RUTA-02`	`ValueError`	M	relaciones (1)	Pasa 1/1
C-19	`cargar`	Pedido huérfano en el CSV	Relación	`CLI-099` inexistente	`ValueError`	A	csv (1)	Pasa 1/1
C-20	`eliminar_cliente` / `_ruta`	No elimina con pedidos asociados	Relación	`CLI-010` y `RUTA-03` con `PED-0001`	`ValueError`; persiste al recargar	A	relaciones (2)	Pasa 2/2
C-21	`registrar_pedido`	Folios consecutivos	Normal	Dos registros desde 0	`PED-0001`, `PED-0002`	A	gestor_crud (1)	Pasa 1/1

ID	Componente	Criterio	Tipo	Escenario y datos	Resultado esperado	Pr.	Prueba (casos)	Resultado
C-22	`eliminar_pedido` + `cargar`	El folio no se reutiliza	Persist.	Registrar, eliminar, recargar, registrar	`PED-0002`; `ultimo_folio == 1`	A	integracion (1)	Pasa 1/1
C-23	`cambiar_estado`	Transiciones válidas persisten	Normal	`pendiente → en_transito → entregado`	`True`; persiste al recargar	A	integracion (1)	Pasa 1/1
C-24	`cambiar_estado` / `cancelar_pedido`	Transiciones prohibidas	Error	5 transiciones prohibidas y cancelar en tránsito	`ValueError`; estado intacto	A	estados (6)	Pasa 6/6
C-25	`eliminar_pedido`	Solo pendientes o cancelados	Error	En tránsito; `PED-9999`	`ValueError`; `False`	M	gestor_crud (2)	Pasa 2/2
C-26	`pendientes_por_transportar`	Solo pendientes y en tránsito	Normal	Pedidos en los 4 estados	Cerrados excluidos; vacío → `[]`	M	gestor_crud (1)	Pasa 1/1
C-27	`rutas_compatibles`	Solo disponibles del mismo trayecto	Normal	Ruta válida, ocupada y de otro trayecto	Solo `RUTA-03`	M	gestor_crud (1)	Pasa 1/1
C-28	`cancelar_pedido`	`pendiente → cancelado` es válido	Normal	`PED-0001` pendiente	`True`; estado `cancelado`	A	estados (1)	Pasa 1/1
C-29	`cambiar_estado`	Estados finales sin salidas	Estado final	`entregado` y `cancelado` hacia los 4 estados	`ValueError`; estado intacto	A	estados (8)	Pasa 8/8
C-30	`cambiar_estado`	Sin ruta no sale a tránsito	Error	Pedido pendiente sin ruta	`ValueError`; sigue pendiente	A	estados (1)	Pasa 1/1
C-31	`cambiar_estado`	Estado o folio inexistente	Error	`perdido`; `PED-9999`	`ValueError`; `False`	M	estados (2)	Pasa 2/2
C-32	`crear_pedido`	Un pedido nuevo nace pendiente	Error	Nace `cancelado`; nace `entregado`	`ValueError`; no se crea	A	gestor_crud (2)	Pasa 2/2
C-33	`cargar`	Rechaza claves duplicadas en un CSV	Error	`CLI-010` dos veces en `clientes.csv`	`ValueError`	A	csv (1)	Pasa 1/1
C-34	`actualizar_ruta`	No cambia el trayecto de una ruta con pedidos	Relación	`RUTA-03` → destino Irapuato	`ValueError`; ruta intacta	A	gestor_crud (1)	Pasa 1/1
C-35	`limpiar_fila`	Normaliza una fila sucia	Limpieza	Espacios, minúsculas, estado vacío	Fila normalizada	A	—	Sin prueba
C-36	`limpiar_fila`	Rechazo informa campo y motivo	Rechazo	Cantidad, ciudad, fecha, folio inválidos	`FilaInvalidaError`	A	—	Sin prueba
C-37	`limpiar_fila`	Estado inicial solo pendiente	Rechazo	`entregado`, `perdido`, vacío	Rechazo o `pendiente`	A	—	Sin prueba
C-38	`cargar_lote`	Cliente inexistente rechaza la fila	Relación	`CLI-099`	Fila rechazada	A	—	Sin prueba
C-39	`cargar_lote`	Ruta existente, del trayecto y disponible	Relación	`RUTA-99`, `RUTA-01`, `RUTA-02`	Tres filas rechazadas	A	—	Sin prueba
C-40	`cargar_lote`	Folio repetido se rechaza	Rechazo	`PED-0001`; `PED-0005` dos veces	Rechazos por duplicado	A	—	Sin prueba
C-41	`cargar_lote`	Una fila mala no detiene el lote	Lote	2 buenas, 3 malas	2 aceptadas, 3 rechazadas	A	—	Sin prueba

ID	Componente	Criterio	Tipo	Escenario y datos	Resultado esperado	Pr.	Prueba (casos)	Resultado
C-42	`cargar_lote`	Encabezado equivocado detiene el lote	Lote	Falta `cantidad`; sobra `piezas`	`LoteInvalidoError`	A	—	Sin prueba
C-43	`cargar_lote`	Archivo vacío o solo encabezado	Lote	0 bytes; una línea	Error; 0 y 0	M	—	Sin prueba
C-44	`cargar_lote`	Reporte con línea y motivo exactos	Rechazo	Rechazo en la 3.ª fila	`linea == 4`	M	—	Sin prueba
C-45	`cargar_lote` + `cargar`	Lo aceptado persiste y el folio avanza	Persist.	`PED-0005`, `PED-0007`	`ultimo_folio == 7`	A	—	Sin prueba
C-46	`cargar_lote`	Cargar dos veces no duplica	Lote	Mismo archivo dos veces	2.ª carga: 0 aceptadas	A	—	Sin prueba
C-47	`cargar_lote`	Fila con columnas de más o de menos	Rechazo	Fila de 7 columnas	Rechazada; lote continúa	M	—	Sin prueba

Fuente: elaboración propia. C-01 a C-27: Acta del Encargado de Pruebas; C-28 a C-31 y C-35 a C-47: Acta del Documentador (renumerados); C-32 a C-34: apartado 3.4. «Sin prueba»: el componente `rutas_bajio/lotes.py` no existe en el repositorio.

Trazabilidad criterio–prueba

La Tabla 3.5 vincula cada criterio con las funciones de prueba que lo cubren. La relación se obtuvo leyendo los docstrings de `tests/` y no a mano, lo que evita que un criterio aparezca como cubierto sin una prueba real.

Tabla 3.5: Funciones de prueba que cubren cada criterio.

ID	Archivo	Función de prueba
C-01	`test_modelos.py`	`test_cliente_clave_mal_formada_lanza_value_error`
C-02	`test_modelos.py`	`test_cliente_telefono_limites`
C-03	`test_validadores.py`	`test_validar_ciudad_normaliza_mayusculas`, `test_validar_ciudad_fuera_del_catalogo_lanza_value_error`
C-04	`test_validadores.py`	`test_entero_positivo_invalido_lanza_value_error`, `test_entero_positivo_convierte_texto_valido`
C-05	`test_modelos.py`	`test_ruta_origen_igual_destino_lanza_value_error`, `test_pedido_origen_igual_destino_lanza_value_error`
C-06	`test_modelos.py`	`test_pedido_en_transito_sin_ruta_lanza_value_error`
C-07	`test_validadores.py`	`test_fecha_iso_invalida_lanza_value_error`
C-08	`test_modelos.py`	`test_to_dict_from_dict_ida_y_vuelta`
C-09	`test_csv.py`	`test_cargar_sin_archivos_arranca_vacio`
C-10	`test_csv.py`	`test_cargar_csv_solo_encabezado_equivale_a_vacio`
C-11	`test_integracion.py`	`test_crear_y_recargar_conserva_registros_y_orden`
C-12	`test_csv.py`	`test_leer_encabezados_incorrectos_lanza_value_error`, `test_leer_fila_incompleta_lanza_value_error`
C-13	`test_csv.py`	`test_guardar_conserva_comas_y_acentos`
C-14	`test_csv.py`	`test_escribir_no_deja_archivos_tmp`
C-15	`test_gestor_crud.py`	`test_crear_cliente_duplicado_lanza_value_error`
C-16	`test_relaciones.py`	`test_crear_pedido_cliente_inexistente_lanza_value_error`
C-17	`test_relaciones.py`	`test_crear_pedido_ruta_de_otro_trayecto_lanza_value_error`, `test_asignar_ruta_de_otro_trayecto_lanza_value_error`
C-18	`test_relaciones.py`	`test_crear_pedido_ruta_no_disponible_lanza_value_error`
C-19	`test_csv.py`	`test_cargar_pedido_con_cliente_inexistente_lanza_value_error`
C-20	`test_relaciones.py`	`test_eliminar_con_pedidos_asociados_lanza_value_error`
C-21	`test_gestor_crud.py`	`test_registrar_pedido_genera_folios_consecutivos`
C-22	`test_integracion.py`	`test_folio_no_se_reutiliza_tras_eliminar_pedido`
C-23	`test_integracion.py`	`test_cambiar_estado_flujo_valido_persiste`
C-24	`test_estados.py`	`test_cambiar_estado_prohibido_lanza_value_error`, `test_cancelar_pedido_en_transito_lanza_value_error`
C-25	`test_gestor_crud.py`	`test_eliminar_pedido_en_transito_lanza_value_error`, `test_eliminar_pedido_inexistente_devuelve_false`
C-26	`test_gestor_crud.py`	`test_pendientes_por_transportista_excluye_cerrados_e_incluye_vacios`
C-27	`test_gestor_crud.py`	`test_rutas_compatibles_solo_disponibles_del_mismo_trayecto`
C-28	`test_estados.py`	`test_cancelar_pedido_pendiente_es_valido`
C-29	`test_estados.py`	`test_estado_final_no_tiene_salidas`
C-30	`test_estados.py`	`test_pedido_sin_ruta_no_sale_a_transito`
C-31	`test_estados.py`	`test_cambiar_estado_inexistente_lanza_value_error`, `test_cambiar_estado_folio_inexistente_devuelve_false`
C-32	`test_gestor_crud.py`	`test_crear_pedido_estado_inicial_no_pendiente_lanza_value_error`
C-33	`test_csv.py`	`test_cargar_claves_duplicadas_en_csv_lanza_value_error`
C-34	`test_gestor_crud.py`	`test_actualizar_ruta_con_pedidos_no_cambia_trayecto`

ID	Archivo	Función de prueba

Fuente: elaboración propia a partir de los docstrings de `tests/`.

Cobertura de la matriz

De los 47 criterios unificados, 34 tienen al menos una prueba que los cubre (72.3 %) y los 34 pasan. Los 27 criterios del Acta del Encargado de Pruebas están cubiertos en su totalidad. De los 30 criterios de prioridad alta, 20 tienen prueba; los 10 restantes pertenecen a la carga por lote (C-35 a C-42, C-45 y C-46), que no puede probarse porque el componente no existe. Ningún criterio de prioridad alta de un componente implementado quedó sin prueba.

3.3 Resultados de ejecución

La suite completa se ejecutó el 5 de noviembre de 2026 con `.venv/bin/python -m pytest -v`. El resultado fue de **71 pruebas pasadas, 0 falladas y 0 omitidas**, en 0.11 segundos. Las 71 pruebas corresponden a 70 casos ligados a criterios más la prueba de humo de `test_entorno.py`. La salida completa está en `docs/evidencias/pytest_completo.txt` y su captura en el Anexo C.

Tabla 3.6: Resultados por marcador.

Ejecución	Comando	Pasadas	Falladas	No seleccionadas
Archivos CSV	`pytest -m csv`	8	0	63
Integración	`pytest -m integracion`	3	0	68
Unitarias (uso diario)	`pytest -m "not integracion"`	68	0	3
Suite completa	`pytest`	71	0	0

Fuente: `docs/evidencias/pytest_marcadores.txt`, *ejecución del 5 de noviembre de 2026.*

La Tabla 3.7 desglosa los resultados por archivo y por nivel de prueba. El nivel *unitario* agrupa las pruebas que construyen entidades sin gestor; el nivel *componente* prueba un método de `GestorRutasBajio` sobre una carpeta temporal; el nivel *integración* encadena varias operaciones y crea un segundo gestor para comprobar que los datos sobreviven a una recarga.

Tabla 3.7: Resultados por archivo y por nivel.

Nivel	Archivo	Clase o componente principal	Pasadas	Fallidas
Unitario	`test_validadores.py`	Validadores de `modelos.py`	12	0
Unitario	`test_modelos.py`	`Cliente`, `Ruta`, `Pedido`	14	0
Componente	`test_csv.py`	`GestorRutasBajio`: `cargar`, `_leer`, `_escribir`	8	0
Componente	`test_gestor_crud.py`	`GestorRutasBajio`: CRUD y consultas	9	0
Componente	`test_relaciones.py`	`GestorRutasBajio`: `_validar_relaciones`, `eliminar_*`	6	0
Componente	`test_estados.py`	`GestorRutasBajio`: `cambiar_estado`, `cancelar_pedido`	18	0
Integración	`test_integracion.py`	Gestor completo con recarga	3	0
Humo	`test_entorno.py`	Fixtures	1	0
Total			**71**	**0**

Fuente: *`docs/evidencias/pytest_por_archivo.txt`.*

Por nivel, las pruebas en verde son 26 de 26 en el nivel unitario, 41 de 41 en el nivel de componente y 3 de 3 en el de integración. Que todas pasen no basta para afirmar que la suite es buena: una prueba sin `assert` o con datos que no ejercitan la regla también pasa. Por ello el siguiente apartado mide si la suite es capaz de fallar cuando el código está mal.

3.4 Calidad de la suite: técnica de mutación

Método

La técnica de mutación consiste en introducir deliberadamente un error pequeño en el código (una *mutación*) y ejecutar la suite: si al menos una prueba falla, la mutación se considera **detectada**; si todas pasan, la mutación **sobrevive** y revela una regla que ninguna prueba protege (Jia y Harman, 2011). Se eligió aplicar las mutaciones a mano, a partir de las reglas de negocio de las actas, en lugar de usar una herramienta automática como `mutmut`, porque una herramienta genera cientos de mutaciones sintácticas sin relación con el negocio y su análisis excedía el tiempo del parcial.

Para no alterar nunca el código del repositorio, se escribió el script `mutaciones.py`. Para cada mutación, el script copia `rutas_bajio/`, `tests/` y `pytest.ini` a una carpeta temporal, reemplaza un único fragmento de código, ejecuta `pytest` en la copia y registra qué pruebas fallaron. El script verifica que el fragmento original aparezca exactamente una vez; si el código cambia y el fragmento ya no existe, el script se detiene en lugar de reportar una mutación que en realidad no se aplicó.

```
fuente = copia / "rutas_bajio" / archivo
codigo = fuente.read_text(encoding="utf-8")
assert codigo.count(original) == 1, \
    f"{ident}: el texto original no aparece exactamente una vez"
```

```
fuente.write_text(codigo.replace(original, mutado), encoding="utf-8")
salida = subprocess.run([sys.executable, "-m", "pytest", "-q",
                         "-p", "no:cacheprovider"],
                        cwd=copia, capture_output=True, text=True)
```

El fragmento anterior muestra el núcleo del script: la mutación se aplica sobre la copia y el código de salida de `pytest` indica si la suite la detectó. Se definieron 14 mutaciones; las cuatro que proponía el Acta del Documentador (folio duplicado, estado inicial distinto de pendiente, transición `en_transito → cancelado` y conteo de líneas del lote) se incluyeron, salvo la del lote, que no tiene código sobre el cual aplicarse.

Resultados

Tabla 3.8: Mutaciones aplicadas y resultado en las dos rondas.

ID	Error inyectado	Ronda 1	Ronda 2	Criterio que la detecta
M-01	`crear_cliente` acepta ids duplicados	Detectada	Detectada	C-15
M-02	Se permite `en_transito → cancelado`	Detectada	Detectada	C-24
M-03	`crear_pedido` acepta un estado inicial distinto de `pendiente`	**Sobrevive**	Detectada	C-32 (nuevo)
M-04	`cargar` ignora `secuencias.csv` (el folio se reutiliza)	Detectada	Detectada	C-22
M-05	Límite del teléfono desplazado (15 dígitos se rechaza)	Detectada	Detectada	C-02
M-06	`entero_positivo` acepta 0	Detectada	Detectada	C-04
M-07	`_validar_relaciones` no compara el trayecto	Detectada	Detectada	C-17
M-08	`_validar_relaciones` ignora la disponibilidad	Detectada	Detectada	C-18
M-09	`eliminar_cliente` borra clientes con pedidos	Detectada	Detectada	C-20
M-10	`cargar` acepta pedidos con cliente inexistente	Detectada	Detectada	C-19
M-11	`cargar` acepta claves duplicadas dentro de un CSV	**Sobrevive**	Detectada	C-33 (nuevo)
M-12	`actualizar_ruta` cambia el trayecto de una ruta con pedidos	**Sobrevive**	Detectada	C-34 (nuevo)
M-13	`Pedido` en tránsito o entregado sin ruta	Detectada	Detectada	C-06, C-30
M-14	`_leer` acepta encabezados incorrectos	Detectada	Detectada	C-12
	Detectadas	**11 de 14**	**14 de 14**	

Fuente: `docs/evidencias/mutaciones_ronda1.txt` y `mutaciones_ronda2.txt`, ejecuciones del 5 de noviembre de 2026.

En la primera ronda, con la suite construida a partir del Acta del Encargado de Pruebas, la suite detectó 11 de las 14 mutaciones (78.6 %). Las tres que sobrevivieron revelaron criterios que faltaban en la matriz:

- **M-03 (estado inicial).** Ninguna prueba intentaba crear un pedido que naciera `cancelado` o `entregado`. La regla existía en el código, pero si alguien la borraba, un pedido podía registrarse como entregado sin haber salido nunca, y la máquina de estados quedaba anulada desde el origen. Se agregó **C-32**.
- **M-11 (claves duplicadas en el CSV).** Las pruebas verificaban que `crear_cliente` rechaza duplicados (C-15), pero no que `cargar` rechace un archivo editado a mano con dos filas del mismo id. Es una diferencia importante: la primera protección actúa sobre la operación, la segunda sobre el archivo. Se agregó **C-33**.
- **M-12 (trayecto de una ruta con pedidos).** Ninguna prueba usaba `actualizar_ruta`. Sin la regla, se podía cambiar el destino de `RUTA-03` y sus pedidos quedarían asignados a una ruta que ya no hace su trayecto, el mismo daño que C-17 evita al crear el pedido. Se agregó **C-34**.

Las tres pruebas nuevas se escribieron en la convención de la suite. En la segunda ronda la suite detectó las 14 mutaciones (100 %). El hallazgo más relevante es que los tres criterios faltantes compartían un patrón: la regla se había probado en *una* de las vías por las que entra un dato, pero no en las demás (creación frente a carga del CSV, creación frente a actualización). Esta observación orientó la revisión del apartado 3.6.

3.5 Defectos encontrados y corregidos

Tabla 3.9: Defectos detectados durante la verificación.

N.º	Defecto	Cómo se detectó	Corrección	Estado
D-1	La suite no protegía el estado inicial del pedido, la unicidad de claves al cargar el CSV ni el trayecto en `actualizar_ruta`	Mutaciones M-03, M-11 y M-12 sobrevivieron	Criterios C-32, C-33 y C-34 con sus pruebas	Corregido
D-2	Las dos actas usaban los mismos identificadores (C-01 a C-19) para criterios distintos	Revisión de las actas al integrar la matriz	Matriz unificada C-01 a C-47 (Tabla 3.3)	Corregido

N.º	Defecto	Cómo se detectó	Corrección	Estado
D-3	`main.py` no se ejecuta: importa `Cliente` desde `rutas_bajio/__init__.py`, que está vacío; importa el módulo `rutas_bajio.consola`, que no existe; y llama a `crear_pedido` con argumentos sueltos, mientras que el gestor recibe un objeto `Pedido`	Ejecución de `python main.py --demo` durante la verificación final (`ImportError`)	Turnado al Desarrollador CRUD: `main.py` corresponde a una versión del código distinta de la del paquete	Abierto

Fuente: elaboración propia.

Ninguna de las 71 pruebas falló sobre el código del paquete `rutas_bajio/`: no se encontraron reglas de negocio implementadas de forma incorrecta. Los defectos D-1 y D-2 pertenecen a la suite y a la documentación, no al código bajo prueba. El defecto D-3 sí afecta al sistema, pero se encuentra fuera de la suite, porque `main.py` no forma parte del paquete que se prueba; se documenta aquí porque se descubrió durante la verificación y porque muestra que una suite en verde no garantiza que el punto de entrada funcione.

3.6 Límites: qué no se probó y por qué

- **Carga por lote (C-35 a C-47).** El Acta del Documentador diseñó 13 criterios para `limpiar_fila` y `cargar_lote`, 10 de ellos de prioridad alta, pero el módulo `rutas_bajio/lotes.py` no se implementó. Los criterios se conservan en la matriz como *Sin prueba* en lugar de omitirse, para que el pendiente quede visible.
- **Métodos sin prueba.** Las consultas `listar_clientes`, `listar_rutas`, `listar_pedidos`, `pedidos_por_cliente`, `pedidos_sin_ruta` y `detalle_pedido`, las actualizaciones `actualizar_cliente` y `actualizar_pedido`, y el validador `decimal_positivo` no tienen prueba directa. Se priorizaron las reglas que, al fallar, dañan datos; estos métodos solo leen o repiten validaciones ya probadas. Aun así, la suite actual **no detectaría**, por ejemplo, que `actualizar_pedido` permitiera corregir un pedido que ya está en tránsito, ni que `listar_pedidos` devolviera pedidos de otro estado.
- **Capacidad de los camiones y concurrencia.** La disponibilidad de una ruta es un indicador manual; no se modeló la capacidad de carga. Tampoco se probó el acceso simultáneo de dos procesos a los mismos CSV, porque el sistema está pensado para un solo usuario.
- **Ejecución en Colab y en el cuaderno.** Se probó el paquete `rutas_bajio/`, copia sin cambios de las celdas del cuaderno; no se verificó automáticamente que el cuaderno y el paquete sigan siendo idénticos.

- **Mutaciones.** Las 14 mutaciones se eligieron a mano a partir de las reglas de negocio. Un 100 % de detección indica que esas reglas están protegidas, no que la suite detecte cualquier error posible.

Con la estrategia de pruebas y sus resultados establecidos, el Capítulo 4 describe cómo se organizó el equipo para producir el sistema, la suite y este reporte.

Referencias del capítulo

Jia, Y. y Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering, 37*(5), 649–678. https://doi.org/10.1109/TSE.2010.62

pytest-dev. (2026). *pytest documentation* (versión 9.1). https://docs.pytest.org/